

✓ 04 — Exploratory neural data analysis

I inspect the preprocessed population response matrix before fitting advanced latent models. I use real visual features extracted from the Allen natural movie pixels, quantify fine/coarse/confident orientation-label imbalance, visualize representative movie frames, and use circular-shift null tests for temporal neural--stimulus correlations.

```
from pathlib import Path
import sys

ROOT = Path.cwd()
if ROOT.name == "notebooks":
    ROOT = ROOT.parent
if str(ROOT / "src") not in sys.path:
    sys.path.insert(0, str(ROOT / "src"))

from v1_manifold.config import load_config, get_paths, set_global_seed
from v1_manifold.visualization import set_publication_style, save_figure
from v1_manifold.utils import save_table

cfg = load_config(ROOT / "configs" / "default.yaml")
cfg["paths"]["root"] = str(ROOT)
paths = get_paths(cfg)
set_global_seed(cfg["project"]["random_seed"])
set_publication_style()
print(f"Project root: {ROOT}")
```

Project root: c:\Users\Peter\Documents\projects\NeuroAI\latent-manifold-v1-na

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import spearmanr, zscore
from v1_manifold.data_access import get_boc, get_experiment_data, get_stimulus
from v1_manifold.preprocessing import load_trial_tensor_h5, repeat_average_pos
from v1_manifold.features import (
    build_frame_feature_table,
    build_real_movie_feature_table,
    fallback_frame_labels,
    assert_real_stimulus_features,
    add_coarse_orientation_target,
    add_confident_orientation_target,
    summarize_class_balance,
    orientation_label_interpretation_table,
```

```

)
from v1_manifold.evaluation import stimulus_neural_circular_shift_table
from v1_manifold.visualization import plot_population_heatmap

h5_files = sorted(paths.interim_dir.glob("session_*_tensor.h5"))
if not h5_files:
    raise FileNotFoundError("No trial tensor exists yet. I need to run notebook")

tensor_path = h5_files[0]
experiment_id = int(tensor_path.name.split("_")[1])
tensor, R = load_trial_tensor_h5(tensor_path)

# I extract real stimulus features from the actual Allen natural movie pixels.
boc = get_boc(paths.allen_manifest)
data_set = get_experiment_data(boc, experiment_id)
movie = get_stimulus_template(data_set, cfg["preprocessing"]["stimulus_name"])

if movie is None:
    if cfg["features"].get("require_real_stimulus_features", True):
        raise RuntimeError(
            "I could not load the Allen natural movie template. Real stimulus-
            "data_set.get_stimulus_template(stimulus_name). I am not falling t
        )
    labels = fallback_frame_labels(R.shape[0], cfg["features"]["orientation_b
else:
    print("Movie template shape:", np.asarray(movie).shape)
    labels = build_real_movie_feature_table(
        movie,
        n_orientation_bins=cfg["features"]["orientation_bins"],
        n_spatial_frequency_bins=cfg["features"]["spatial_frequency_bins"],
        downsample_factor=cfg["features"].get("stimulus_downsample_factor", 4),
        max_frames=R.shape[0],
    )

population_features = build_frame_feature_table(R, labels=None)
features = population_features.merge(labels, on="movie_frame", how="left")

# This assertion prevents placeholder labels from leaking into final decoding
assert_real_stimulus_features(features)

# I add coarse and confidence-aware orientation targets because hard natural-r
# orientation bins can be imbalanced and ambiguous.
features = add_coarse_orientation_target(
    features,
    top_k=cfg["features"].get("coarse_orientation_top_k", 2),
)
features = add_confident_orientation_target(
    features,
    confidence_quantile=cfg["features"].get("orientation_confidence_quantile",
    ambiguous_label=cfg["features"].get("orientation_ambiguous_label", "ambigu

```

```

/

base_name = tensor_path.name.replace("_tensor.h5", "")
save_table(features, paths.processed_dir / f"{base_name}_real_frame_features.csv")
# I also save the canonical filename used by downstream notebooks.
save_table(features, paths.processed_dir / f"{base_name}_frame_features.csv")

print("Real stimulus-derived features saved for", len(features), "movie frames")
display(features.head())

```

```

c:\Users\Peter\neuro\Lib\site-packages\allensdk\core\brain_observatory_nwb_data_reader.py
from pkg_resources import parse_version
Movie template shape: (900, 304, 608)
Real stimulus-derived features saved for 900 movie frames

```

	movie_frame	population_mean	population_std	population_l2_norm	n_cells
0	0	0.459886	2.380123	30.949413	163
1	1	0.475428	2.378806	30.971182	163
2	2	0.436005	2.304510	29.943974	163
3	3	0.373745	2.170050	28.113255	163
4	4	0.420181	2.040942	26.603487	163

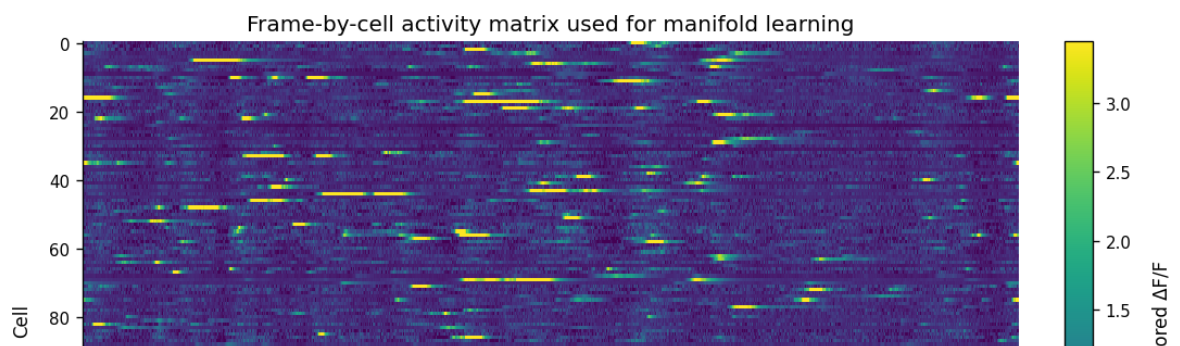
5 rows × 34 columns

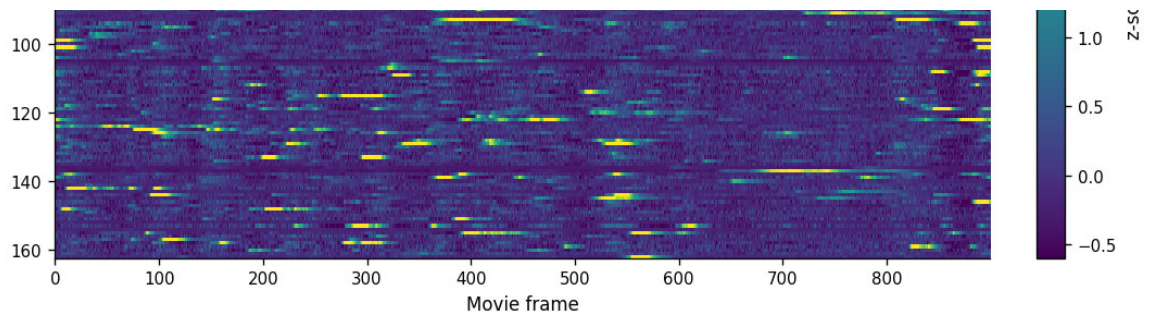
```

# Robust clipping is only for visualization. The saved tensor and downstream
R_plot = R.copy()
vmin, vmax = np.nanpercentile(R_plot, [1, 99])

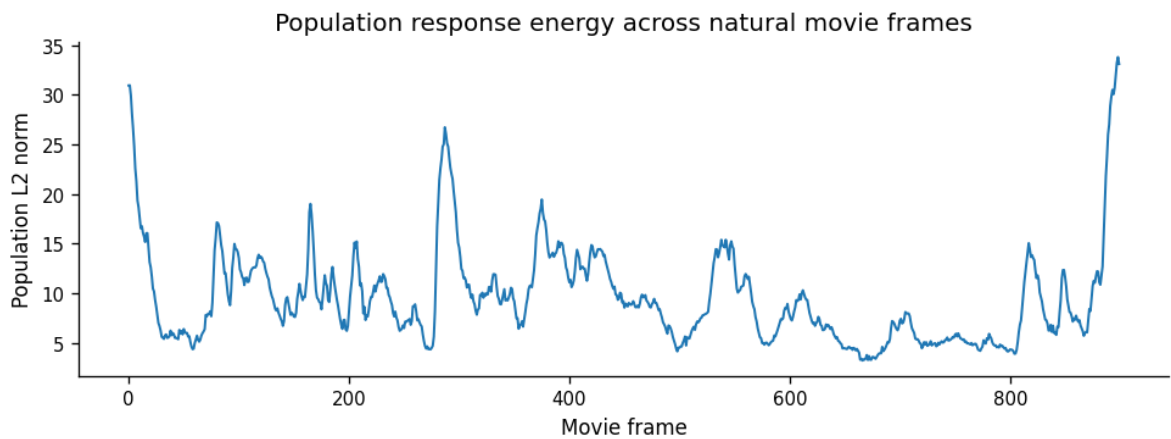
fig, ax = plt.subplots(figsize=(9, 5))
im = ax.imshow(R_plot.T, aspect="auto", interpolation="nearest", vmin=vmin,
ax.set_xlabel("Movie frame")
ax.set_ylabel("Cell")
ax.set_title("Frame-by-cell activity matrix used for manifold learning")
cbar = fig.colorbar(im, ax=ax)
cbar.set_label("z-scored  $\Delta F/F$ ")
save_figure(fig, paths.figures_dir / "04_population_activity_heatmap_robust.
plt.show()

```





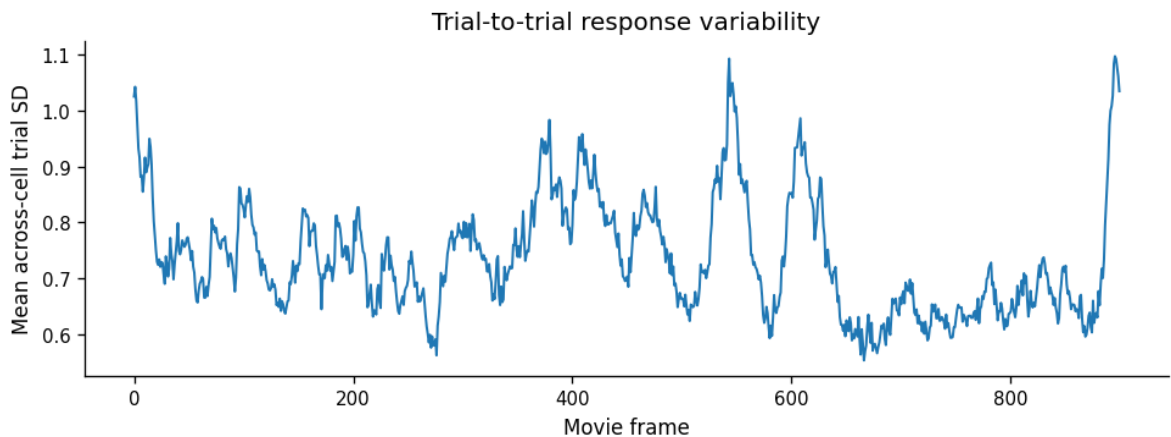
```
fig, ax = plt.subplots(figsize=(8, 3))
ax.plot(features["movie_frame"], features["population_l2_norm"], linewidth=1)
ax.set_xlabel("Movie frame")
ax.set_ylabel("Population L2 norm")
ax.set_title("Population response energy across natural movie frames")
save_figure(fig, paths.figures_dir / "04_population_response_energy.png")
plt.show()
```



```

trial_variability = tensor.std(axis=0).mean(axis=0)
fig, ax = plt.subplots(figsize=(8, 3))
ax.plot(np.arange(len(trial_variability)), trial_variability, linewidth=1.2)
ax.set_xlabel("Movie frame")
ax.set_ylabel("Mean across-cell trial SD")
ax.set_title("Trial-to-trial response variability")
save_figure(fig, paths.figures_dir / "04_trial_variability_by_frame.png")
plt.show()

```



```

fig, axes = plt.subplots(1, 3, figsize=(12, 3.2))
axes[0].plot(features["movie_frame"], features["rms_contrast"], linewidth=1)
axes[0].set_title("RMS contrast")
axes[0].set_xlabel("Movie frame")
axes[0].set_ylabel("Contrast")

axes[1].plot(features["movie_frame"], features["spatial_frequency_centroid"])
axes[1].set_title("Spatial-frequency centroid")
axes[1].set_xlabel("Movie frame")
axes[1].set_ylabel("Frequency centroid")

axes[2].hist(features["dominant_orientation_bin"], bins=cfg["features"]["ori"])
axes[2].set_title("Dominant orientation bins")
axes[2].set_xlabel("Orientation bin")
axes[2].set_ylabel("Frames")

fig.tight_layout()

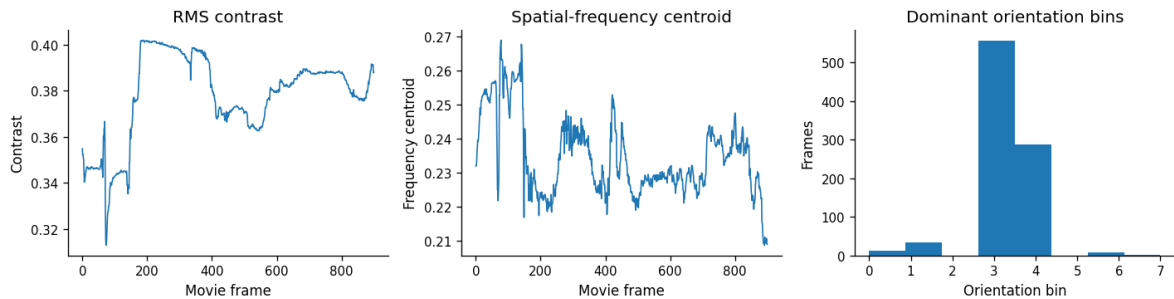
```

```

save_figure(fig, paths.figures_dir / "04_real_movie_frame_features.png")
plt.show()

```

C:\Users\Peter\AppData\Local\Temp\ipykernel_35344\3293651419.py:17: UserWarning
fig.tight_layout()



```

orientation_counts = summarize_class_balance(
    features["dominant_orientation_bin"],
    target_name="dominant_orientation_bin",
)
save_table(orientation_counts, paths.tables_dir / "04_dominant_orientation_b

coarse_orientation_counts = summarize_class_balance(
    features["dominant_orientation_coarse"],
    target_name="dominant_orientation_coarse",
)
save_table(coarse_orientation_counts, paths.tables_dir / "04_dominant_orient

confident_orientation_counts = summarize_class_balance(
    features["dominant_orientation_confident"],
    target_name="dominant_orientation_confident",
)
save_table(confident_orientation_counts, paths.tables_dir / "04_confident_or

orientation_interpretation = orientation_label_interpretation_table(features
save_table(orientation_interpretation, paths.tables_dir / "04_orientation_la

display(orientation_counts)
display(coarse_orientation_counts)
display(confident_orientation_counts)

```

```

display(orientation_interpretation)

fig, axes = plt.subplots(1, 3, figsize=(13.5, 3.4))

axes[0].bar(
    orientation_counts["dominant_orientation_bin"].astype(str),
    orientation_counts["n_samples"],
)
axes[0].set_xlabel("Dominant orientation bin")
axes[0].set_ylabel("Number of movie frames")
axes[0].set_title("Fine orientation-label balance")

axes[1].bar(
    coarse_orientation_counts["dominant_orientation_coarse"].astype(str),
    coarse_orientation_counts["n_samples"],
)
axes[1].set_xlabel("Coarse orientation class")
axes[1].set_ylabel("Number of movie frames")
axes[1].set_title("Coarse orientation-label balance")

axes[2].bar(
    confident_orientation_counts["dominant_orientation_confident"].astype(str),
    confident_orientation_counts["n_samples"],
)
axes[2].set_xlabel("Confidence-aware orientation class")
axes[2].set_ylabel("Number of movie frames")
axes[2].set_title("Orientation labels after ambiguity filtering")
axes[2].tick_params(axis="x", rotation=30)

fig.tight_layout()
save_figure(fig, paths.figures_dir / "04_orientation_class_balance.png")
plt.show()

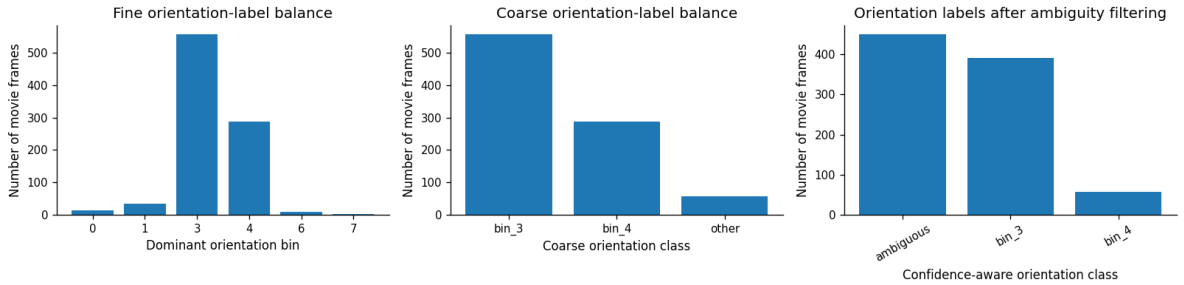
```

	dominant_orientation_bin	n_samples	fraction	majority_class
0	0	13	0.014444	False
1	1	33	0.036667	False
2	3	557	0.618889	True
3	4	287	0.318889	False
4	6	8	0.008889	False
5	7	2	0.002222	False
	dominant_orientation_coarse	n_samples	fraction	majority_class
0	bin_3	557	0.618889	True
1	bin_4	287	0.318889	False

2	other	56	0.062222	False
dominant_orientation_confident n_samples fraction majority_class				
0	ambiguous	450	0.500000	True
1	bin_3	392	0.435556	False
2	bin_4	58	0.064444	False

finding		value
0	n_fine_orientation_classes	6
1	fine_orientation_majority_fraction	0.618889
2	n_coarse_orientation_classes	3
3	coarse_orientation_majority_fraction	0.618889
4	fraction_ambiguous_orientation_frames	0.5
5	dominant_confident_orientation_class	bin_3
6	minority_confident_orientation_class	bin_4
7	recommended_primary_decoding_target	continuous visual features
8	recommended_orientation_decoding_status	secondary/exploratory with class balance, ambi...

C:\Users\Peter\AppData\Local\Temp\ipykernel1_35344\1194532002.py:54: UserWarning: fig.tight_layout()



```
# I visually audit the orientation labels by sampling frames across each con
# This makes it clear that the labels are derived from real movie pixels, wh
```

```

# this makes it clear that the labels are derived from real movie pixels, with
# showing why hard orientation classification is secondary for natural movie
if movie is not None:
    movie_array = np.asarray(movie)
    classes = list(confident_orientation_counts["dominant_orientation_confid
n_examples = 4
    fig, axes = plt.subplots(len(classes), n_examples, figsize=(10, 2.0 * le
    if len(classes) == 1:
        axes = np.asarray([axes])

    for row_idx, cls in enumerate(classes):
        frame_ids = features.loc[
            features["dominant_orientation_confident"].astype(str) == cls,
            "movie_frame",
        ].astype(int).values

        if len(frame_ids) == 0:
            for col_idx in range(n_examples):
                axes[row_idx, col_idx].axis("off")
            continue

        selected_idx = np.linspace(0, len(frame_ids) - 1, min(n_examples, le
        selected = frame_ids[selected_idx]

        for col_idx in range(n_examples):
            ax = axes[row_idx, col_idx]
            if col_idx >= len(selected):
                ax.axis("off")
                continue
            frame_id = int(selected[col_idx])
            ax.imshow(movie_array[frame_id], cmap="gray")
            ax.set_title(f"{cls}, frame {frame_id}")
            ax.axis("off")

    fig.suptitle("Representative natural movie frames by confidence-aware or
    fig.tight_layout()
    save_figure(fig, paths.figures_dir / "04_example_movie_frames_by_confide
    plt.show()

```

C:\Users\Peter\AppData\Local\Temp\ipykernel_35344\3093733387.py:37: UserWarning:
fig.tight_layout()

Representative natural movie frames by confidence-aware orientation class





bin_4, frame 75



bin_4, frame 395



bin_4, frame 418



bin_4, frame 443



```
stimulus_cols = [
    "rms_contrast",
    "spatial_frequency_centroid",
    "orientation_selectivity",
    "luminance_std",
    "total_spectral_power",
]

# I report the ordinary Spearman coefficient, but I use a circular-shift nul
# because adjacent natural movie frames are temporally autocorrelated.
ordinary_rows = []
for target in stimulus_cols:
    rho, p = spearmanr(
        features["population_l2_norm"],
        features[target],
        nan_policy="omit",
    )
    ordinary_rows.append({
        "neural_feature": "population_l2_norm",
        "stimulus_feature": target,
        "spearman_r": rho,
        "ordinary_framewise_p": p,
    })
```

```

ordinary_corr = pd.DataFrame(ordinary_rows)

shift_corr = stimulus_neural_circular_shift_table(
    features,
    neural_col="population_l2_norm",
    stimulus_cols=stimulus_cols,
    n_perm=cfg["evaluation"].get("circular_shift_permutations", 1000),
    min_shift=cfg["evaluation"].get("circular_shift_min_frames", 60),
    seed=cfg["project"]["random_seed"],
)

stimulus_neural_checks = ordinary_corr.merge(
    shift_corr,
    on=["neural_feature", "stimulus_feature", "spearman_r"],
    how="left",
)

save_table(
    stimulus_neural_checks,
    paths.tables_dir / "04_neural_population_energy_vs_real_stimulus_feature"
)

display(stimulus_neural_checks)

```

	neural_feature	stimulus_feature	spearman_r	ordinary_framewise_p
0	population_l2_norm	rms_contrast	-0.109517	9.989034e-04
1	population_l2_norm	spatial_frequency_centroid	-0.012127	7.163669e-01
2	population_l2_norm	orientation_selectivity	-0.248246	4.169368e-14
3	population_l2_norm	luminance_std	-0.107345	1.258901e-03
4	population_l2_norm	total_spectral_power	0.070242	3.512260e-02

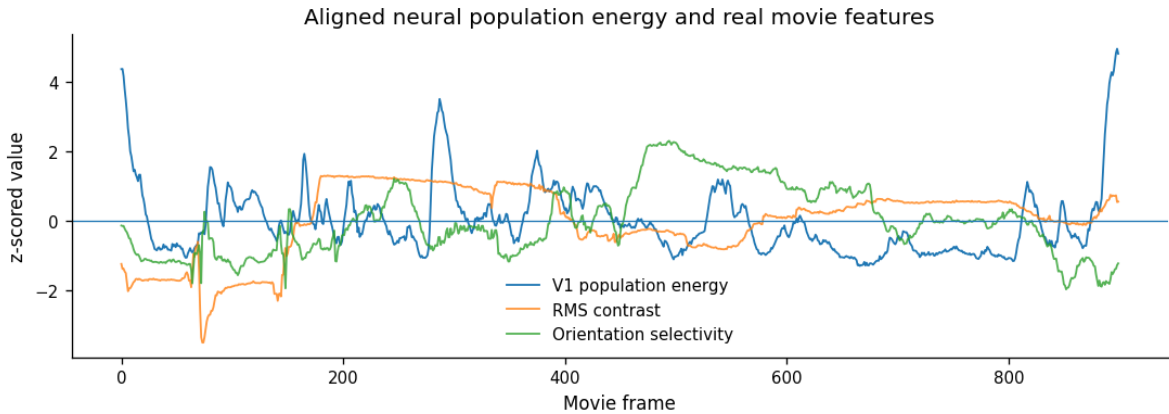
```

plot_df = features[["movie_frame", "population_l2_norm", "rms_contrast", "or
plot_df["population_l2_norm_z"] = zscore(plot_df["population_l2_norm"], nan_
plot_df["rms_contrast_z"] = zscore(plot_df["rms_contrast"], nan_policy="omit
plot_df["orientation_selectivity_z"] = zscore(plot_df["orientation_selectivi

fig, ax = plt.subplots(figsize=(9, 3.2))
ax.plot(plot_df["movie_frame"], plot_df["population_l2_norm_z"], linewidth=1
ax.plot(plot_df["movie_frame"], plot_df["rms_contrast_z"], linewidth=1.1, al
ax.plot(plot_df["movie_frame"], plot_df["orientation_selectivity_z"], linewi
ax.axhline(0, linewidth=0.8)
ax.set_xlabel("Movie frame")
ax.set_ylabel("z-scored value")
ax.set_title("Aligned neural population energy and real movie features")
ax.legend(frameon=False)

```

```
save_figure(fig, paths.figures_dir / "04_zscored_population_energy_and_movie
plt.show()
```



```
eda_summary = pd.DataFrame({
    "metric": [
        "n_movie_frames",
        "n_cells",
        "mean_population_l2_norm",
        "std_population_l2_norm",
        "max_population_l2_norm",
        "mean_trial_variability",
        "max_trial_variability",
        "uses_fallback_labels",
        "mean_rms_contrast",
        "mean_spatial_frequency_centroid",
        "n_fine_orientation_classes",
        "fine_orientation_majority_fraction",
        "n_coarse_orientation_classes",
        "coarse_orientation_majority_fraction",
        "n_confident_orientation_classes",
        "confident_orientation_majority_fraction",
        "fraction_ambiguous_orientation_frames",
    ],
    "value": [
        R.shape[0],
        R.shape[1],
        features["population_l2_norm"].mean(),
        features["population_l2_norm"].std(),
```

```

features["population_l2_norm"].max(),
trial_variability.mean(),
trial_variability.max(),
bool(features["fallback_labels"].any()),
features["rms_contrast"].mean(),
features["spatial_frequency_centroid"].mean(),
orientation_counts.shape[0],
orientation_counts["fraction"].max(),
coarse_orientation_counts.shape[0],
coarse_orientation_counts["fraction"].max(),
confident_orientation_counts.shape[0],
confident_orientation_counts["fraction"].max(),
confident_orientation_counts.loc[
    confident_orientation_counts["dominant_orientation_confident"].a
    "fraction",
].sum(),
],
})
save_table(eda_summary, paths.tables_dir / "04_exploratory_summary.csv")
display(eda_summary)

```

	metric	value
0	n_movie_frames	900
1	n_cells	163
2	mean_population_l2_norm	9.554914
3	std_population_l2_norm	4.916242
4	max_population_l2_norm	33.812862
5	mean_trial_variability	0.735869
6	max_trial_variability	1.097426
7	uses_fallback_labels	False
8	mean_rms_contrast	0.377728
9	mean_spatial_frequency_centroid	0.234498
10	n_fine_orientation_classes	6
11	fine_orientation_majority_fraction	0.618889
12	n_coarse_orientation_classes	3
13	coarse_orientation_majority_fraction	0.618889
14	n_confident_orientation_classes	3
15	confident_orientation_majority_fraction	0.5

```
save_table(  
    features,  
    paths.processed_dir / f"{base_name}_frame_features.csv",  
)  
  
save_table(  
    features,  
    paths.processed_dir / f"{base_name}_real_frame_features.csv",  
)
```

```
WindowsPath('C:/Users/Peter/Documents/projects/NeuroAI/latent-manifold-v1-  
natural-movies/data/processed/  
session_500855614_natural_movie_one_real_frame_features.csv')
```